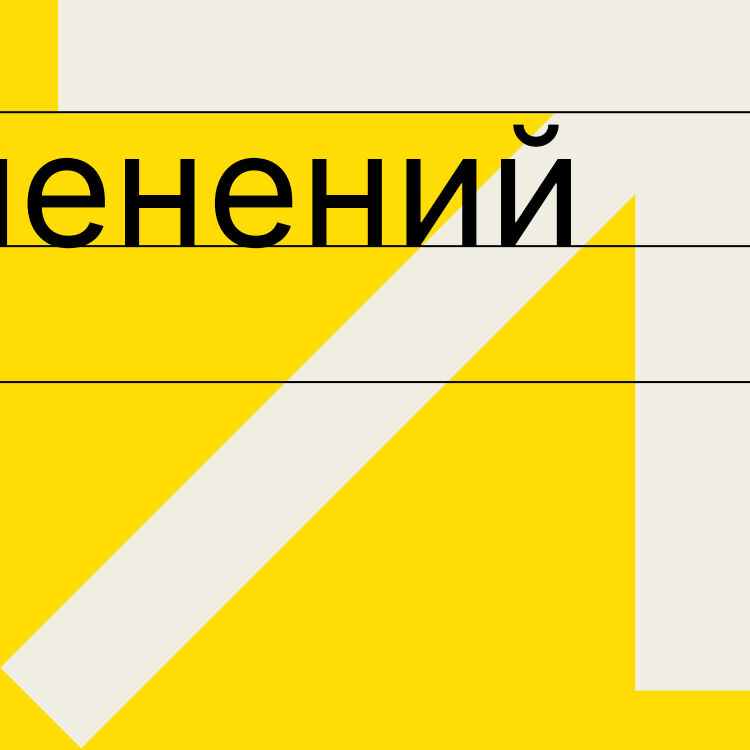


Разработка в условиях изменений требований

An abstract geometric design on a yellow background. It features several white shapes: a horizontal rectangle in the top right, a vertical rectangle on the right side, and a large diagonal rectangle extending from the middle right towards the bottom left.

Предметная область и постановка задачи

Проблема **Продукт** Задача проекта

- авторские маршруты по городам существуют как статичный контент: блоги, подборки, карты
- нет продукта, где маршрут можно опубликовать, продать, пройти с навигацией и получить обратную связь
- платформа «впечатлений» - маршрутов с упорядоченными точками, покупкой и прохождением по геолокации
- разработать backend MVP: API полного жизненного цикла контента - от черновика автора до отзыва туриста
- создать роли: user/author/admin
- маршруты, точки, впечатления, витрина
- realtime прохождение
- интеграция с геопровайдером Яндекс

Техническое задание: структура и масштаб

Состав ТЗ

- термины, сценарии пользователя / автора / администратора
- роли и общие правила доступа
- общие правила данных и валидации текстов
- 12 функциональных доменов: для каждого - состав данных, пример JSON, валидации, API-методы
- общие требования к API: формат ответа, коды ошибок, пагинация и сортировка
- нефункциональные требования и требования к проверке качества

В цифрах

- 44 API-метода (REST + WebSocket)
- 3 роли, 5 статусов контента
- 8 сущностей данных
- единый конверт ответа {status, data, error}
- сортировка только по белым спискам полей
- 31 обязательный сценарий проверки качества

Ключевые проектные решения в ТЗ

Статусная модель контента

draft → on_moderation → published / rejected (+ archived); автор публикует без модерации, при правках published возвращается в draft

Realtime-прохождение

WebSocket; расстояние по гаверсинусу; точка засчитывается в радиусе 30 м; завершение открывает право на отзыв

Рекомендации без ML

$\text{score} = \text{rating} \times 2 + \text{popularity}$; жёсткие фильтры: город, цена, длительность

Единый контракт ошибок

401 / 403 / 404 / 409 / 422; бизнес-конфликты — всегда 409 с понятным сообщением

Реальность: стек и архитектура

Стек

- FastAPI (async) + Pydantic v2 — typed API и валидация
- SQLAlchemy 2 async + PostgreSQL (docker-compose)
- JWT — stateless-авторизация
- pytest + httpx — интеграционные тесты

Слои

- API-роутеры по доменам (13 модулей)
- схемы → модели → сервисный слой → инфраструктура
- общие бизнес-правила не дублируются между доменами

Реалии

- все 44 метода из таблицы ТЗ реализованы
- финальная сверка ТЗ и кода по каждому методу

Жизненный цикл



1. Маршрут + точки (draft)

2. Submit → модерация

3. Впечатление (цена)

4. Витрина / рекомендации

5. Покупка → оплата

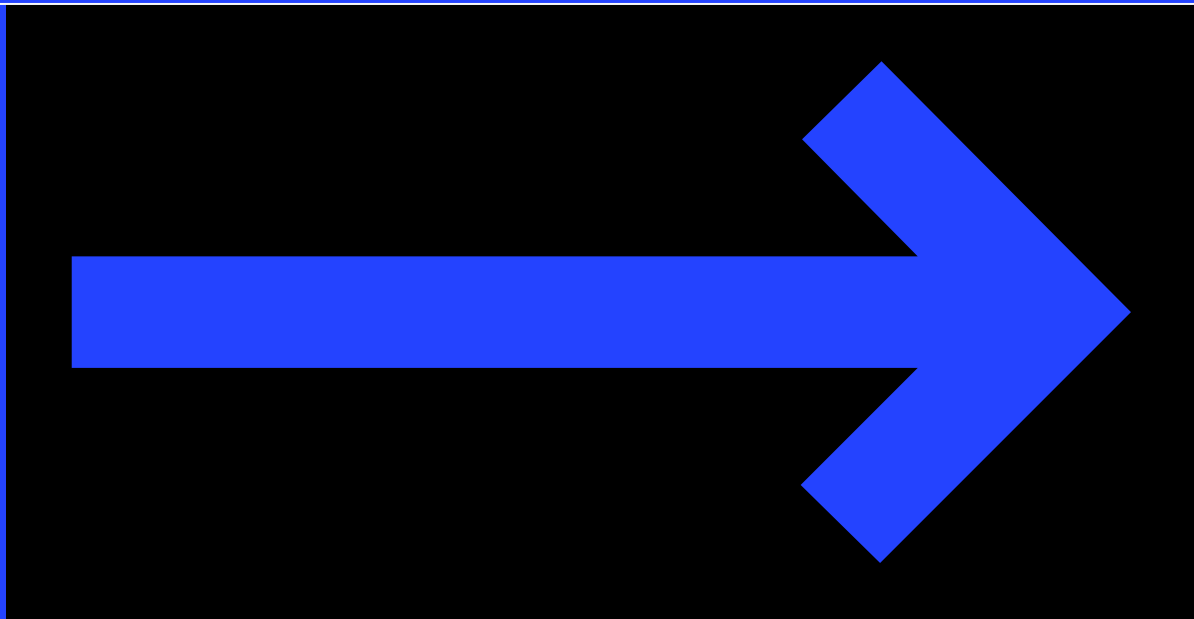
6. WS-прохождение по точкам

7. Отзыв → пересчёт рейтинга

8. Аналитика → популярность

Service status			^
GET	/health	Health	▼
Authentication			^
POST	/auth/register	Register	▼
POST	/auth/login	Login	▼
Users and administration			^
GET	/users/me	Me	🔒 ▼
PATCH	/admin/users/{user_id}/block	Block User	🔒 ▼
PATCH	/admin/users/{user_id}/verify-author	Verify Author	🔒 ▼
Routes			^
POST	/routes	Create Route	🔒 ▼
GET	/routes	List Routes	🔒 ▼
GET	/routes/{route_id}	Get Route	🔒 ▼
PATCH	/routes/{route_id}	Update Route	🔒 ▼
DELETE	/routes/{route_id}	Delete Route	🔒 ▼
POST	/routes/{route_id}/submit	Submit Route	🔒 ▼
GET	/moderation/routes	Moderation Routes	🔒 ▼
POST	/moderation/routes/{route_id}/publish	Publish Route	🔒 ▼
POST	/moderation/routes/{route_id}/reject	Reject Route	🔒 ▼
POST	/routes/{route_id}/points	Create Point	🔒 ▼
GET	/routes/{route_id}/points	List Points	🔒 ▼
PATCH	/routes/{route_id}/points/{point_id}	Update Point	🔒 ▼
DELETE	/routes/{route_id}/points/{point_id}	Delete Point	🔒 ▼

Демонстрация



Как проверял, что задача решена

01

Интеграционные тесты (pytest + TestClient + PostgreSQL)

02

Самостоятельная проверка через фронтенд/сваггер, пытаюсь представить себя на месте пользователя, вызываю специально ошибки

03

Понимаю, что задача решена как только полностью покрыв код техническое задание

Рефлексия: ожидание / реальность

Ожидание

- ТЗ заранее опишет всё поведение системы
- BFF-экран прохождения - отдельный метод (был в ТЗ)
- основная работа - написать эндпоинты

Вывод: ТЗ == живой документ. Финальная сверка «ТЗ → код → тесты» самый обязательный шаг, а не формальность

Реальность

- первый же реальный клиент нашёл пробелы: CORS, список городов витрины, имена в ответах покупок и отзывов
- оказался лишним: экран собирается из карточки впечатления и API прогресса - требование убрано
- основная сложность - инварианты: статусы, права, условия submit; именно эти места ТЗ пришлось доописывать по итогам реализации